01_PROJECT_SETUP.md

TASK 01

PROJECT INITIALIZATION

Status

Mandatory

Owner

Project Director

Primary Engineer

Full Stack Engineering Team

Estimated Time

2 to 4 Hours

Dependencies

00_PROJECT_CONTEXT.md

00_ENGINEERING_RULES.md

00_PROJECT_MEMORY.md

01_PROJECT_DIRECTOR.md

Mission

Initialize a production-ready foundation for the Secure Dance Academy Management System.

Do not build business features.

Do not create placeholder code.

Create the engineering foundation that every future feature will depend on.

Every decision made during setup should improve maintainability, security and scalability.

Objectives

Create a professional project structure.

Configure development tools.

Configure security foundations.

Configure code quality tools.

Prepare the application for future development.

Ensure every engineer starts from the same baseline.

Read Before Starting

Read and understand:

Project Context

Engineering Rules

Project Memory

Project Director Handbook

Do not begin implementation until these documents have been reviewed.

Deliverables

Create

Next.js 15 Application

TypeScript Configuration

Tailwind CSS

shadcn/ui

Prisma

Supabase Connection

Environment Variable Template

Docker Configuration

Git Repository

README

License

Ignore Files

Project Documentation Folder

Testing Folder

Architecture Folder

Assets Folder

Public Folder

Folder Structure

Create a clean feature-first architecture.

Example

app/

features/

components/

lib/

services/

repositories/

hooks/

types/

utils/

middleware/

prisma/

tests/

docs/

public/

scripts/

config/

Avoid unnecessary folders.

Every folder must have a clear responsibility.

Install Dependencies

Install only approved packages.

Frontend

Next.js

Tailwind CSS

shadcn/ui

React Hook Form

Zod

Backend

Prisma

Supabase

Validation

Zod

Authentication

Supabase Auth

Utilities

clsx

tailwind-merge

Testing

Jest

Playwright

ESLint

Prettier

Avoid unnecessary packages.

Every dependency should have a clear purpose.

Configure Development Environment

Configure

TypeScript

ESLint

Prettier

EditorConfig

Tailwind

Path Aliases

Environment Variables

Git Ignore

Docker

Docker Compose

Prisma

Supabase

Configuration should support long-term development.

Create Environment Template

Create a secure example environment file.

Include

Database URL

Supabase URL

Supabase Keys

Application URL

Environment

Never include real credentials.

Never commit secrets.

Configure Authentication Foundation

Prepare authentication.

Do not build login yet.

Configure

Supabase Client

Server Client

Middleware

Protected Routes

Session Handling

Cookie Configuration

Authentication should be ready for implementation.

Configure Database Foundation

Initialize

Prisma

Schema

Migration Folder

Seed Folder

Database Connection

Do not create business tables yet.

Configure UI Foundation

Install

shadcn/ui

Global Layout

Theme

Typography

Color Palette

Responsive Breakpoints

Navigation Placeholder

Notification System

Loading Components

Error Components

Empty State Components

Create reusable foundations.

Configure Testing Foundation

Prepare

Jest

Playwright

Test Folder Structure

Mock Configuration

Coverage Configuration

Testing Utilities

No business tests yet.

Configure Security Foundation

Prepare

Middleware

Protected Routes

Security Headers

Environment Validation

Error Handling

Request Validation

Rate Limiting Structure

Logging Structure

Audit Structure

No feature implementation yet.

Configure Documentation

Create

README

Architecture Folder

API Folder

Security Folder

Testing Folder

Deployment Folder

Decision Records

Documentation should grow with the project.

Configure Git

Initialize repository.

Create

Main Branch

Development Branch

Git Ignore

Commit Template

Repository should be production ready.

Create Placeholder Modules

Generate empty modules for

Authentication

Users

Artists

Parents

Coaches

Attendance

Performance

Injuries

Activities

Dashboard

Reports

Notifications

Settings

Audit

No business logic.

Only structure.

Verify Setup

Confirm

Application runs.

No build errors.

No TypeScript errors.

No lint errors.

No dependency conflicts.

Project structure is clean.

Environment variables load correctly.

Prisma connects successfully.

Supabase configuration is valid.

Update Project Memory

Mark

Project Setup

Completed

Record

Packages Installed

Architecture Decisions

Folder Structure

Configuration Decisions

Known Issues

Next Task

Requirements Engineering

Deliverables Checklist

Next.js Project

Folder Structure

Dependencies

Prisma

Supabase

Environment Template

Docker

Git

Testing Configuration

Security Foundation

Documentation Structure

README

Development Ready

Definition of Done

The project starts successfully.

No configuration errors exist.

The project follows the approved architecture.

Development standards are enforced.

Security foundations are configured.

Testing foundations are configured.

Documentation structure exists.

The project is ready for Requirements Engineering.

Do not begin feature development until every setup task has been completed and verified.